

ФЕДЕРАЛЬНОЕ ГОСУДАРСТВЕННОЕ АВТОНОМНОЕ ОБРАЗОВАТЕЛЬНОЕ УЧРЕЖДЕНИЕ
ВЫСШЕГО ОБРАЗОВАНИЯ

«Национальный исследовательский университет ИТМО»

Факультет программной инженерии и компьютерной техники

«Алгоритмы и структуры данных»

Отчет по блоку задач №4 (Яндекс.Контест)

Выполнил:

Студент группы Р3209

Евграфов А. А.

Преподаватель:

Косяков М. С.

Санкт-Петербург, 2026 г.

Задачи из 4-ого блока Яндекс.Контеста

Задача №13 «М. Цивилизация»

Описание

Карта мира представляет собой прямоугольник $N \times M$ ($N, M \leq 1000$). Клетки имеют разные типы рельефа: поле (.) - прохождение стоит 1 единицу времени, лес (W) - 2 единицы времени, вода (#) - непреодолима. Заданы начальные и конечные координаты поселенца. На каждом шаге можно перемещаться в одну из 4-х соседних клеток (по вертикали или горизонтали). Требуется найти минимальное время для достижения цели и вывести маршрут в виде последовательности символов: N (вверх), S (вниз), E (вправо), W (влево).

Ограничение времени: 1.5 секунды.

Ограничение памяти: 256 Мб.

Решение

Применяем алгоритм Дейкстры для нахождения кратчайшего пути во взвешенном графе. Граф представляет собой сетку, где каждая клетка — вершина, рёбра соединяют соседние клетки с весами, зависящими от типа рельефа. Используем приоритетную очередь для эффективного выбора вершины с минимальным расстоянием.

Доказательство

Алгоритм Дейкстры находит кратчайший путь из начальной вершины во все остальные. Инвариант алгоритма: после извлечения вершины из приоритетной очереди расстояние до неё — окончательное и минимальное.

Предположим, что существует более короткий путь от стартовой вершины s до вершины v с расстоянием $d' < d[v]$, где $d[v]$ — расстояние, найденное алгоритмом. Пусть u — первая вершина на этом пути, расстояние до которой ещё не финализировано, когда мы обработали v . Тогда расстояние до u должно быть не больше d' , так как оно лежит на более коротком пути. Но алгоритм всегда обрабатывает вершину с минимальным расстоянием; если $d[u] \leq d'$, а $d' < d[v]$, то u должен был быть обработан до v . Это противоречие доказывает, что алгоритм находит оптимальное решение. Восстановление пути выполняется через массив родителей, где запоминается, из какой клетки мы пришли в каждую позицию.

Алгоритмическая сложность

- **По времени:** $O(N \cdot M \cdot \log(N \cdot M))$ Приоритетная очередь содержит $O(N \cdot M)$ элементов в худшем случае. Каждая вершина извлекается один раз, каждое ребро обрабатывается один раз, операции с приоритетной очередью — $O(\log(N \cdot M))$.
- **По памяти:** $O(N \cdot M)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int dr[] = {-1, 1, 0, 0};
6 int dc[] = {0, 0, -1, 1};
7 char dir_chars[] = {'N', 'S', 'W', 'E'};
8
9 int main() {
10     ios_base::sync_with_stdio(false);
11     cin.tie(nullptr);
12
13     int n, m;
14     cin >> n >> m;
15
16     int sr, sc, er, ec;
17     cin >> sr >> sc >> er >> ec;
18
19     --sr;
20     --sc;
21     --er;
22     --ec;
23
24     vector<string> grid(n);
25     for (int i = 0; i < n; ++i) {
26         cin >> grid[i];
27     }
28
29     vector<vector<int>> dist(n, vector<int>(m, 1e9));
30     vector<vector<char>> parent(n, vector<char>(m, 0));
31
32     using State = pair<int, pair<int, int>>;
33     priority_queue<State, vector<State>, greater<State>> pq;
34
35     dist[sr][sc] = 0;
36     pq.push(make_pair(0, make_pair(sr, sc)));
37
38     while (!pq.empty()) {
39         auto top = pq.top();
40         pq.pop();
41
42         int d = top.first;
43         int r = top.second.first;
44         int c = top.second.second;
45
46         if (d > dist[r][c]) {
47             continue;
48         }
49
50         if (r == er && c == ec) {
51             break;
52         }
53
54         for (int i = 0; i < 4; ++i) {
55             int nr = r + dr[i];
56             int nc = c + dc[i];
57
58             if (nr >= 0 && nr < n && nc >= 0 && nc < m && grid[nr][nc] != '#') {
```

```

59     int cost = (grid[nr][nc] == 'W') ? 2 : 1;
60     if (dist[r][c] + cost < dist[nr][nc]) {
61         dist[nr][nc] = dist[r][c] + cost;
62         parent[nr][nc] = dir_chars[i];
63         pq.push(make_pair(dist[nr][nc], make_pair(nr, nc)));
64     }
65 }
66 }
67 }
68
69 if (dist[er][ec] == 1e9) {
70     cout << -1 << "\n";
71 } else {
72     cout << dist[er][ec] << "\n";
73
74     string path = "";
75     int r = er;
76     int c = ec;
77     while (r != sr || c != sc) {
78         char p = parent[r][c];
79         path += p;
80         if (p == 'N') {
81             r++;
82         } else if (p == 'S') {
83             r--;
84         } else if (p == 'W') {
85             c++;
86         } else if (p == 'E') {
87             c--;
88         }
89     }
90
91     reverse(path.begin(), path.end());
92     cout << path << "\n";
93 }
94
95 return 0;
96 }

```

Задача №14 «N. Свинки-копилки»

Описание

У Васи есть n свинок-копилков ($1 \leq n \leq 100$). Каждая копилка может быть открыта ключом или разбита. Ключи спрятаны внутри самих копилков, причем известно, в какой именно копилке лежит ключ от каждой из них. Васе нужно достать деньги из всех копилков, разбив при этом как можно меньшее их количество.

Ограничение времени: 1 секунда.

Ограничение памяти: 256 Мб.

Решение

Задача сводится к нахождению циклов в графе, где каждая копилка i имеет ребро в копилку, в которой лежит ключ от копилки i . Каждая вершина имеет ровно одно исходящее ребро. Копилку нужно разбить, если она содержится в цикле. Используем DFS с отслеживанием стека рекурсии: при нахождении обратного ребра (в вершину, находящуюся в текущем пути) мы обнаружили цикл.

Доказательство

Используем метки: -1 (не посещена), 1 (в текущем пути DFS), 2 (полностью обработана). Когда из вершины со статусом 1 есть ребро в другую вершину со статусом 1 , это означает обратное ребро (цикл). Счётчик подсчитывает количество таких обнаруженных циклических рёбер, что равно количеству циклов.

Алгоритмическая сложность

- **По времени:** $O(n)$ Каждая вершина посещается один раз, каждое ребро обрабатывается один раз.
- **По памяти:** $O(n)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 int c = 0;
6
7 void dfs(int i, vector<vector<int>>& v, vector<int>& mark) {
8     mark[i] = 1;
9     for (int j : v[i]) {
10         if (mark[j] == -1) {
11             dfs(j, v, mark);
12         } else if (mark[j] == 1) {
13             c++;
14         }
15     }
```

```

16  mark[i] = 2;
17  }
18
19  int main() {
20      ios_base::sync_with_stdio(false);
21      cin.tie(nullptr);
22
23      int n;
24      cin >> n;
25      vector<vector<int>> v(n + 1);
26      int x;
27
28      for (int i = 1; i <= n; i++) {
29          cin >> x;
30          v[x].push_back(i);
31      }
32
33      vector<int> mark(n + 1, -1);
34
35      for (int i = 1; i <= n; i++) {
36          if (mark[i] == -1) {
37              dfs(i, v, mark);
38          }
39      }
40
41      cout << c;
42      return 0;
43  }

```

Задача №15 «О. Долой списывание!»

Описание

Заданы N учеников ($1 \leq N \leq 100$) и M пар учеников, которые обменивались записками. Требуется определить, можно ли разделить всех учеников на две непересекающиеся группы («списывающие» и «дающие списывать») так, чтобы любой обмен записками происходил строго между представителями разных групп. Если такое разделение возможно, вывести YES, иначе NO.

Ограничение времени: 2 секунды.

Ограничение памяти: 64 Мб.

Решение

По сути мы хотим проверить является ли граф двудольным. Граф двудольный тогда и только тогда, когда его можно раскрасить двумя цветами так, чтобы никакие две соседние вершины не имели одинаковый цвет. Используем DFS для раскраски: начинаем с произвольной непометенной вершины, назначаем ей цвет 1, всем её соседям — цвет -1 , и продолжаем рекурсивно. Если при этом обнаружим ребро между вершинами одного цвета, граф не двудольный.

Доказательство

Граф содержит только чётные циклы (является двудольным) тогда и только тогда, когда он раскрашивается двумя цветами.

Необходимость: если граф двудольный и разбит на две группы, то ребра соединяют только вершины из разных групп. Значит, цикл любой длины должен поочередно переходить между группами и, чтобы вернуться в исходную группу, пройти чётное число шагов.

Достаточность: если граф раскрашивается двумя цветами, то по определению раскраски никакие две соседние вершины не одного цвета, следовательно, граф содержит только чётные циклы.

Алгоритм DFS корректно проверяет раскрашиваемость: начиная с произвольной вершины, мы присваиваем ей цвет и поочередно чередуем цвета для всех достижимых вершин. Если при обработке ребра мы обнаруживаем, что обе его концы имеют одинаковый цвет, это означает нечётный цикл, и граф не двудольный.

Алгоритмическая сложность

- **По времени:** $O(N + M)$ Каждая вершина посещается один раз, каждое ребро проверяется один раз.
- **По памяти:** $O(N + M)$ Граф хранится в виде списка смежности, массив раскраски требует $O(N)$ памяти.

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 bool flag = true;
6
7 void dfs(int i, vector<vector<int>>& v, vector<int>& mark, int col) {
8     mark[i] = col;
9     for (int j : v[i]) {
10         if (mark[j] == 0) {
11             dfs(j, v, mark, -col);
12         } else if (mark[j] == col) {
13             flag = false;
14             return;
15         }
16     }
17 }
18
19 int main() {
20     ios_base::sync_with_stdio(false);
21     cin.tie(nullptr);
22
23     int n, m;
24     cin >> n >> m;
25
26     vector<vector<int>> v(n + 1);
27     int x, y;
28
29     for (int i = 0; i < m; i++) {
30         cin >> x >> y;
31         v[x].push_back(y);
32         v[y].push_back(x);
33     }
34
35     vector<int> mark(n + 1, 0);
36
37     for (int i = 1; i <= n; i++) {
38         if (mark[i] == 0) {
39             dfs(i, v, mark, 1);
40         }
41     }
42
43     if (flag) {
44         cout << "YES";
45     } else {
46         cout << "NO";
47     }
48
49     return 0;
50 }
```


Задача №16 «Р. Авиаперелёты»

Описание

Дана матрица $N \times N$ ($1 \leq N \leq 1000$), где элемент $c_{i,j}$ ($0 \leq c_{i,j} \leq 10^9$) обозначает расход топлива на перелёт из города i в город j . Требуется найти минимально возможный размер топливного бака самолёта, при котором из любого города можно будет долететь в любой другой (возможно, совершая дозаправки в промежуточных городах). Иными словами, нужно найти минимальный вес максимального ребра, при котором граф, состоящий только из рёбер с весом не больше заданного, является сильно связным.

Ограничение времени: 2 секунд.

Ограничение памяти: 256 Мб.

Решение

Бинарим по ответу. Если мы можем долететь из города i в город j с баком размера x , то мы можем это сделать и с баком размера $x + 1$. Граф с лимитом топлива x содержит только рёбра, вес которых не превышает x . Граф сильно связан при данном лимите, если из любого города можно дойти до любого другого.

Для проверки сильной связности используем два DFS: из вершины 0 в прямом графе (проверяем, что все вершины достижимы из стартовой) и из вершины 0 в обратном графе (проверяем, что стартовая вершина достижима из всех остальных). Если оба DFS посещают все вершины, граф сильно связан.

Доказательство

Бинарный поиск работает благодаря монотонности: множество значений x , при которых граф сильно связан, образует отрезок $[x_{min}, \infty)$. Множество всех весов рёбер конечно, поэтому оптимальный ответ совпадает с одним из весов рёбер.

Проверка сильной связности: граф сильно связан тогда и только тогда, когда из любой вершины достижима любая другая. Эквивалентно: из вершины 0 достижимы все вершины И из всех вершин достижима вершина 0 (через обратный граф). Два DFS проверяют оба условия за $O(N^2)$ при работе с матрицей смежности.

Бинарный поиск выполняет $O(\log(\max c_{i,j}))$ итераций, каждая из которых требует $O(N^2)$ для проверки. Таким образом, в худшем случае ответ равен максимальному весу ребра, при котором граф становится сильно связным.

Алгоритмическая сложность

- По времени: $O(N^2 \log(\max c_{i,j}))$
- По памяти: $O(N^2)$

Код

```
1 #include <bits/stdc++.h>
2
3 using namespace std;
4
5 void dfs(int v, vector<vector<int>>& e, vector<bool>& mark, int n, int x) {
6     mark[v] = true;
7     for (int i = 0; i < n; i++) {
8         if (i != v && e[v][i] <= x && !mark[i]) {
9             dfs(i, e, mark, n, x);
10        }
11    }
12 }
13
14 int main() {
15     ios_base::sync_with_stdio(false);
16     cin.tie(nullptr);
17
18     int n;
19     cin >> n;
20
21     int x;
22     vector<vector<int>> e(n);
23     vector<vector<int>> rev(n, vector<int>(n));
24     vector<bool> mark1(n, false);
25     vector<bool> mark2(n, false);
26
27     int r = 0;
28
29     for (int i = 0; i < n; i++) {
30         for (int j = 0; j < n; j++) {
31             cin >> x;
32             r = max(r, x);
33             e[i].push_back(x);
34             rev[j][i] = x;
35         }
36     }
37
38     int l = -1;
39
40     while (l + 1 < r) {
41         int m = l + (r - l) / 2;
42         bool flag = true;
43
44         dfs(0, e, mark1, n, m);
45         dfs(0, rev, mark2, n, m);
46
47         for (int i = 0; i < n; i++) {
48             flag &= mark1[i];
49             flag &= mark2[i];
50             mark1[i] = false;
51             mark2[i] = false;
52         }
53
54         if (flag) {
55             r = m;
56         } else {
57             l = m;
58         }
59     }
```

```
59 }  
60  
61 cout << r;  
62 return 0;  
63 }
```